C++ OOP

Important Concepts, Rules & Code Patterns

Revision notes based on our System Design / OOP discussions

Basics • Access Modifiers • Constructors • Inheritance • Overloading • Overriding • Encapsulation • Abstraction • Polymorphism

Goal: A practical revision sheet for the concepts and common mistakes we discussed while moving from Java OOP to C++.

Keep this PDF beside your code notes. The examples are compact; compile and modify them to test your understanding.

1. Classes, Objects & Multiple Classes

Class: a blueprint defining data (data members) and behavior (member functions). **Object:** an instance of a class.

```
#include <iostream>
#include <string>
using namespace std;

class Car {
public:
    string brand;

    void drive() {
        cout << brand << " is moving\\n";
    }
};

int main() {
    Car c;             // object
    c.brand = "Toyota";
    c.drive();
}
```

• A class can be declared in one file and its member-function definitions can be written elsewhere.

• With multiple classes, one object can contain another object as a data member (composition).

• For example, a Car object may own an Engine object. When the Car is constructed, its Engine member is constructed too; when the Car is destroyed, its Engine member is destroyed automatically.

• A member object is constructed before the containing class constructor body runs, and destroyed after the containing class destructor body finishes.

2. Access Modifiers

Modifier	Accessible inside same class	Accessible in derived class	Accessible through object from outside
public	Yes	Yes (subject to inheritance mode)	Yes
protected	Yes	Yes (subject to inheritance mode)	No
private	Yes	No, not directly	No

Important: Being in the same file does not grant special access. Access is controlled by the class and inheritance rules, not by the file location.

C++ default access: class members are private by default; struct members are public by default.

```
class Example {
    int x;          // private by default
public:
    int y;
};
```

3. Getters, Setters & Encapsulation

Encapsulation bundles data and functions together and controls how an object's state can be accessed or changed. A common pattern is to keep data private and expose public methods.

```
class BankAccount {
private:
    double balance = 0;

public:
    double getBalance() const {
        return balance;
```

```
    }

    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
};
```

• A getter returns a value; a setter or operation changes it.

• You do not need a setter for every private field. Expose only operations that make sense.

• Validation inside methods helps keep the object in a valid state.

• The const after a getter means the member function promises not to modify the object's ordinary member state.

4. The this Keyword

`this` is a pointer to the current object inside a non-static member function. It is often used when a parameter has the same name as a member.

```
class Student {
private:
    int age;

public:
    Student(int age) {
        this->age = age;
    }
};
```

Here, `this->age` means the object's member; plain `age` refers to the constructor parameter.

You may also see `: age(age)` in a member-initializer list; that is generally preferred for initializing members.

5. Constructors

A constructor initializes an object. Its name matches the class name, and it has no return type—not even void.

Default vs parameterized

```
class Student {
public:
    string name;
    int age;

    Student() : name("Unknown"), age(0) {} // default constructor

    Student(string name, int age)
        : name(name), age(age) {}           // parameterized
};
```

• A default constructor can be called with no arguments. If you declare your own constructor, the compiler does not necessarily generate the default constructor for you.

• A member-initializer list initializes members before the constructor body runs.

• Prefer member-initializer lists for members that need initialization; they avoid default-initialize-then-assign behavior.

• The order of member initialization follows the order of member declarations in the class, not the order written in the initializer list.

Composition and constructor order

```
class Engine {
public:
    Engine() { cout << "Engine constructed\\n"; }
    ~Engine() { cout << "Engine destroyed\\n"; }
};
```

```
class Car {
    Engine engine; // member object
public:
    Car() { cout << "Car constructed\\n"; }
    ~Car() { cout << "Car destroyed\\n"; }
};
```

Construction: **Engine** → **Car constructor body**. Destruction: **Car destructor body** → **Engine**.

Common C++ vs Java difference

C++ does not use a `super(...)` call in the constructor. Initialize the direct base class in the initializer list, e.g. `Student(...) : Human(name, age)`.

6. Destructors

A destructor runs when an object is destroyed. It is named with a tilde, has no return type, takes no parameters, and cannot be overloaded.

```
class Student {
public:
    Student() { cout << "Constructed\\n"; }
    ~Student() { cout << "Destroyed\\n"; }
};
```

• For ordinary local objects, destruction happens when they leave scope.

• For inheritance, the derived destructor body runs before the base destructor body.

• For member objects, the containing object's destructor body runs first; then members are destroyed in reverse declaration order.

• If a base class is intended to be deleted through a base pointer, its destructor should normally be virtual.

7. Inheritance

Inheritance lets a derived class reuse and extend a base class. It commonly models an **is-a** relationship.

```
class Human {
protected:
    string name;
    int age;

public:
    Human(string name, int age)
        : name(name), age(age) {}
};

class Student : public Human {
private:
    int rollNo;

public:
    Student(string name, int age, int rollNo)
        : Human(name, age), rollNo(rollNo) {}
};
```

• A derived class directly initializes its direct base class—not an ancestor several levels above it.

• If a base class has only parameterized constructors and no default constructor, the derived constructor must initialize that base explicitly.

• If the base has an accessible default constructor, it is called automatically when no base initializer is written.

• In multilevel inheritance, each class initializes its own direct base: the child initializes the parent, and the parent initializes the grandparent.

• Private base members still exist in the derived object, but the derived class cannot access them directly. Use accessible base methods or protected/public members as appropriate.

Inheritance modes

Inheritance syntax	Base public becomes	Base protected becomes
public Base	public	protected
protected Base	protected	protected
private Base	private	private

Private base members remain inaccessible directly in the derived class regardless of inheritance mode.

8. Method Overloading

Overloading means using the same function name with different parameter lists. It is resolved at compile time.

```
class Printer {
public:
    void print(int x) { cout << x << "\\n"; }
    void print(string x) { cout << x << "\\n"; }
    void print(int x, int y) { cout << x + y << "\\n"; }
};
```

• Overloads must differ in their parameter list (number, types, or order).

• Changing only the return type is not enough to overload a function.

• Overloading is compile-time polymorphism.

9. Method Overriding

Overriding means a derived class provides its own implementation of a virtual base-class function with a matching signature.

```
class Human {
public:
    virtual void work() {
        cout << "Human works\\n";
    }

    virtual ~Human() = default;
};

class Student : public Human {
public:
    void work() override {
        cout << "Student studies\\n";
    }
};

int main() {
    Student s;
    Human* ptr = &s;
    ptr->work(); // Student studies
}
```

• `virtual` in the base enables runtime dispatch through a base pointer or reference.

• `override` asks the compiler to verify that the function actually overrides a virtual base function. It is a safety check, not a keyword that hides the implementation.

• Without virtual dispatch, calling through a base pointer normally uses the base version even when the pointed-to object is derived.

• Calling `s.work()` directly on a Student object calls Student's version either way.

10. Polymorphism

Type	How it works	Example
Compile-time	The compiler chooses a matching overload.	Function overloading; operator overloading
Runtime	The actual object type selects the overridden virtual function.	Base pointer/reference calling a derived override

The key runtime pattern is: **base-class pointer/reference + derived object + virtual function**.

```
Student s;
Human& ref = s;
ref.work(); // Student::work() because work() is virtual
```

11. Encapsulation vs Abstraction

Concept	Main idea	Typical C++ technique
Encapsulation	Bundle state and behavior; control access to state.	private data + public methods
Abstraction	Expose what a client needs while hiding implementation details.	Public interface; abstract classes; separate implementation files

These concepts are related but not identical. Private fields help enforce encapsulation; abstraction is about providing a simpler interface that clients can use without depending on internal details.

12. Abstraction: virtual vs pure virtual

Normal virtual function

```
class Human {
public:
    virtual void work() {
        cout << "Default work\\n";
    }
};
```

A normal virtual function supports overriding and runtime dispatch, but the class is still instantiable unless another rule prevents it.

Pure virtual function

```
class Human {
public:
    virtual void work() = 0;
    virtual ~Human() = default;
};

class Student : public Human {
public:
    void work() override {
        // Real implementation details would live here.
        cout << "Student is studying\\n";
    }
};

int main() {
    Student s;
    Human* ptr = &s;
    ptr->work();
}
```

- `= 0` makes the function pure virtual; Human becomes an abstract class and cannot be instantiated directly.
- A concrete derived class must implement all inherited pure virtual functions before it can be instantiated.

• A pure virtual function expresses a required interface: what the class must provide, not the complete implementation.

• The derived implementation is not automatically invisible to everyone. Abstraction means client code can use the interface without needing to know the implementation; in real projects, implementation details are often separated into source files or private implementation components.

> **Quick distinction:** `virtual` enables runtime dispatch; `= 0` makes the function pure virtual and the class abstract (if it has no other way to remain concrete).

13. Four Pillars of OOP — Quick Revision

Pillar	Remember this	Example
Encapsulation	Protect/control object state.	Private balance + deposit()
Abstraction	Show the interface, hide unnecessary implementation detail.	Abstract Human with work() = 0
Inheritance	Reuse/extend a base class; is-a relationship.	Student : public Human
Polymorphism	One interface, different behavior.	virtual work() overridden in Student

14. Common Mistakes to Avoid

• Do not assume class members are public by default. In a class, they are private by default.

• Do not try to initialize an inherited member directly in a derived constructor's initializer list if that member belongs to the base. Initialize the base class instead.

• Do not write `super(...)` in C++; use the base class name in the initializer list.

• Do not expect a base pointer to call the derived override unless the base function is virtual.

• Do not confuse `override` with `virtual`: override checks correctness; virtual enables dynamic dispatch.

• Do not assume every virtual function makes a class abstract. A pure virtual function (e.g. `= 0`) is the key concept.

• Do not try to overload destructors: destructors cannot have parameters and cannot be overloaded.

• Do not assume protected members are accessible through any object from main; protected is not public.

• Do not assume same-file code bypasses access modifiers.

• Remember construction order: base classes before derived class; members before constructor body. Destruction reverses that order.

15. One-Page Cheat Sheet

Question	Answer
Default access in class?	private
Default access in struct?	public
Constructor return type?	None
Destructor name?	~ClassName()
Can destructor be overloaded?	No
C++ equivalent of super constructor call?	BaseClass(args) in initializer list
What enables runtime polymorphism?	Virtual function called through base pointer/reference

Question	Answer
What does override do?	Compile-time check that the function overrides a virtual base function
What makes a class abstract?	At least one pure virtual function not implemented for that class
Derived destructor vs base destructor?	Derived first, then base
Encapsulation vs abstraction?	Control access to state vs expose a simple interface

16. Suggested Practice Checklist

- Create a BankAccount class with private balance and a validated deposit method.
- Create Engine and Car classes; print constructor/destructor order.
- Create Human → Student inheritance with parameterized constructors.
- Demonstrate public, protected, and private access with small compile tests.
- Write three overloaded functions and test how the compiler selects each.
- Call an overridden virtual function through both a base pointer and a base reference.
- Try instantiating an abstract class, then implement its pure virtual function in a derived class.
- Add a virtual destructor to a base class and test deletion through a base pointer.

End of notes. Keep practicing by writing the examples from memory and explaining why each output occurs.